

Protocollo I2C e SPI

Il protocollo I2C un protocollo di comunicazione seriale, sincrono e half-duplex. È progettato per la comunicazione a breve distanza. Distingue in device Master, che iniziano una comunicazione e device slave. Supporta il multi-master cioè più device master possono essere connessi allo stesso bus.

Utilizza due soli segnali:

- SDA (serial Data)
- SCL(serial clock)

Lo stato di riposo delle linee è il valore logico alto. Esiste un protocollo di arbitraggio del bus perché poiché possono esserci più master, questi ultimi pilotano il bus SOLO quando devono cominciare una connessione.

Il limite principale è la velocità di trasmissione.

Ogni device è identificato da un indirizzo a 7bit: l'indirizzo è assegnato direttamente in fase di produzione (si trova ad esempio sul retro del display). Non è modificabile ma alcune periferiche ne possono avere due che possono essere cambiati (vedi dopo).

L'stm agisce come master ovvero genera il segnale di clock e inizia la comunicazione. Il display che utilizza I2C agisce come slave.

La comunicazione inizia quando la linea SDA viene forzata bassa mentre il clock SCL è a livello alto (bit di start)

Il master poi porta SCL a livello basso.

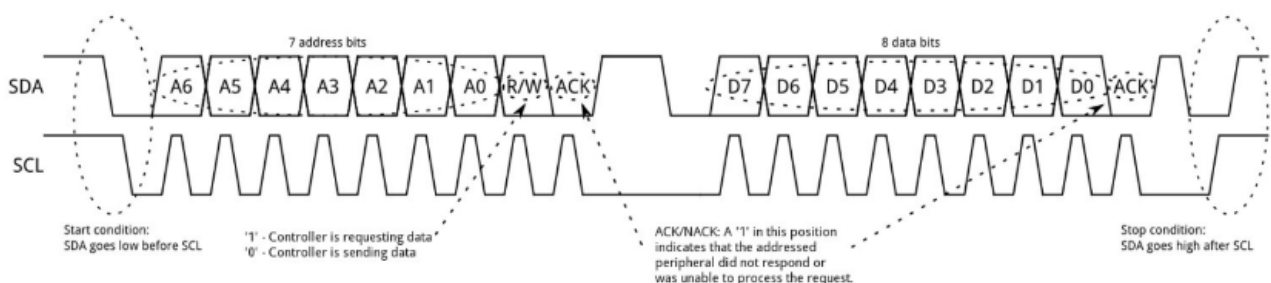
Il master impone sul segnale SDA il valore del primo Bit e appena il clock commuta indica che il bit è stabile e può essere letto. A questo punto si riabbassa il clock e si invia il secondo bit. Quindi il bit viene inviato con il clock basso dal master e letto dallo slave quando il clock è basso. [letto sul clock alto](#)

La comunicazione termina con il bit di STOP ovvero SDA viene commutato da basso ad alto quando il clock è alto. Quindi i bit di start e stop vengono imposti con il clock alto dal master, mentre i dati con il clock basso.

È sempre il master ad aprire la comunicazione sia che vuole scrivere che se vuole leggere.

Dopo lo START il master impone sul bus l'indirizzo dello slave con cui vuole comunicare poi invia un bit che indica se vuole scrivere oppure riceve informazioni.

Se lo slave indirizzato esiste, prende il controllo della linea dati SDA forzandola bassa al successivo clock alto così che il master capisce che lo slave è pronto.



Protocollo di arbitraggio:

- Ogni master controlla il bus per i bit di start e stop e non inizia un messaggio mentre un altro master sta tenendo il bus occupato

- Ogni trasmettitore controlla il livello della linea dati SDA e lo confronta con i livelli che si aspetta e se non corrispondono, quel trasmettitore ha perso l'arbitrato.

Conflitto:

- Due master abbassano SDA per fare lo START. Entrambi leggono 0 sul bus e pensano: "Perfetto, il bus è mio, inizio a trasmettere l'indirizzo".
- **I bit uguali:** Iniziano a inviare i bit in modo sincrono. Finché inviano gli stessi bit (es. entrambi vogliono inviare 0, o entrambi vogliono inviare 1), la linea si comporta esattamente come si aspettano.
- **Il momento della verità:** Arriviamo a un bit in cui i due master differiscono. Supponiamo che il Master A voglia inviare un 1 e il Master B voglia inviare un 0.
 - Il Master A "rilascia" la linea, aspettandosi che torni a 1.
 - Il Master B collega la linea a massa, forzandola a 0.
- **La sconfitta:** Poiché il Master B sta fisicamente cortocircuitando la linea a massa, lo stato reale del bus SDA è 0. Il Master A, prima di procedere, rilegge il bus per sicurezza e nota la discrepanza: *"Io ho rilasciato la linea per inviare 1, ma leggo 0! Qualcun altro sta trasmettendo."* In quel preciso istante, il Master A **perde l'arbitrato** e si ammutolisce istantaneamente. Il Master B, invece, voleva inviare 0, rilegge 0, non si accorge di nulla e continua la sua trasmissione come se fosse l'unico sul bus.

Quindi il primo nodo che nota che esiste una differenza è quello che perde l'arbitrato: smette di pilotare SDA e se è un master smette di pilotare anche SCL.

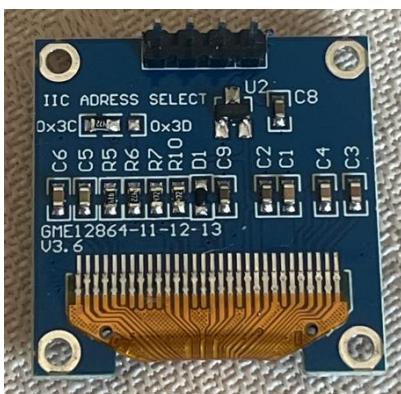
Clock Stretching

Un dispositivo slave può tenere la linea di clock bassa dopo aver ricevuto o inviato un byte indicando che non è ancora pronto ad elaborare altri dati (poiché il master invia con il clock alto quindi non può inviare altri). È l'unico momento in cui lo slave pilota SCL

Caratteristica	I2C	SPI (Serial Peripheral Interface)	USART (Universal Synchronous/Asynchronous)
Numero di fili	2 (SDA, SCL)	4 (MOSI, MISO, SCK, CS)	2 (TX, RX) + GND condiviso
Modalità	Half-Duplex	Full-Duplex	Full-Duplex
Sincronizzazione	Sincrono (Usa clock SCL)	Sincrono (Usa clock SCK)	Asincrono (Nessun clock condiviso)

Caratteristica	I2C	SPI (Serial Peripheral Interface)	USART (Universal Synchronous/Asynchronous)
Velocità	Media (Standard 100 kHz, Fast 400 kHz)	Molto alta (Spesso > 10 MHz)	Bassa/Media (es. 115200 baud)
Topologia	Multi-Master, Multi-Slave	Single-Master, Multi-Slave	Point-to-Point (generalmente 1 a 1)
Selezione Dispositivo	Indirizzo hardware trasmesso sui dati	Linea hardware dedicata (Chip Select - CS)	Indirizzamento software (se implementato)

Se provassimo a collegare lo stesso filo SDA e SCL della board a due display otterremo un conflitto: entrambi avranno lo stesso indirizzo I2C e il master non saprà distinguerli. Entrambi leggeranno quell'indirizzo e risponderanno contemporaneamente e inoltre mostreranno anche lo stesso contenuto.



Il display nella parte di dietro ha due indirizzi: 0x3C e 0x3D. inoltre sono presenti 3 contatti metallici (pad).

Nel chip c'è un PIN specifico in base al quale il livello logico di questo determina l'ultimo bit dell'indirizzo I2C con cui il display si presenterà sul bus.

Vediamo che dove c'è l'indirizzo 0x3C c'è una piccola resistenza tra i due pad a sinistra e quindi collega quel pin specifico a massa.

Se spostiamo la resistenza sull'altro lato, questo pin viene collegato all'alimentazione e quindi il display assumerà il secondo indirizzo.

Nelle librerie HAL vengono utilizzati gli indirizzi a 8 bit, quindi bisogna shiftare di una posizione a sinistra tutto l'indirizzo.

0x3C= 0011 1100

Se shift a sinistra diventa

0111 1000= 0x78

Mentre 0x3D → 0x7A

SPI è un'interfaccia di comunicazione seriale sincrono sviluppato da Motorola, è ancora uno standard per la velocità. È un protocollo di tipo Master-slave full-duplex:

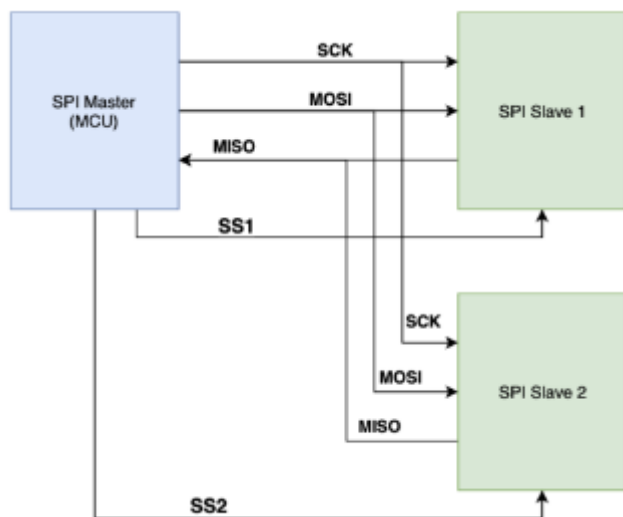
- Master: dispositivo che ha il controllo totale della comunicazione
- Slave: dispositivo che risponde solo se interpellato

Il protocollo si basa su 4 pin:

- MOSI (Master out Slave in) trasmette i dati dal master allo slave
- MISO (master in Slave out) trasmette i dati dallo slave al master
- SCK (Serial Clock) fornisce il segnale di sincronizzazione temporale gestito dal master.
- SS/CS (slave select/Chip select) serve a selezionare lo slave con cui si vuole comunicare

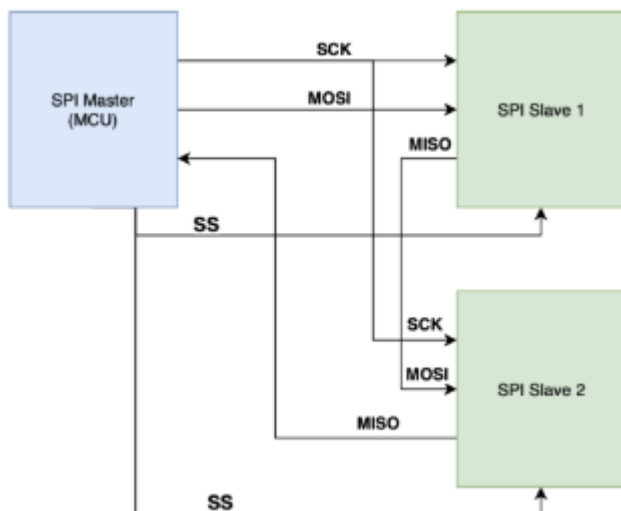
SPI supporta la connessione di un singolo Master a molteplici slave e può avvenire con una connessione parallela o daisy-chain:

- Parallela: I canali MISO, MOSI, SCK sono collegati insieme e poi è presente un canale SS per dispositivo.



Quando bisogna comunicare con uno slave la corrispondente linea SS viene alzata.

- Daisy chain: implica collegare in cascata le linee MISO/MOSI. Tutti gli slave sono coinvolti in una comunicazione. Pertanto è necessario prevedere un indirizzo per il destinatario. I segnali di clock e ss sono condivisi.



Come vediamo il master invia al primo slave. Lo slave è collegato ad un altro slave quindi riceve sulla MOSI da parte della MISO del primo slave. È lo slave 2 che poi ha il MISO collegato al master.

Se il master vuole comunicare con uno slave che non sia il primo, deve inviare per forza al primo e poi per cascata arriva allo slave giusto. Di contro se uno slave manda la risposta non può inviarla al master, ma sempre per catena, la passa agli slave e l'ultimo la manda al master.